

**Full Name:** Kevin Korcari

**Course Name and Code:** IA445 22001

**Instructor's Name:** Che

**Title:** (Penetration Testing of OWASP Juice Shop ) Project Full Report

## **Penetration Testing of OWASP Juice Shop**

### **Abstract**

This project presents a comprehensive penetration test of the OWASP Juice Shop web application conducted in a sandboxed Docker environment using Parrot OS. The objective was to identify vulnerabilities, analyze their impact, and demonstrate ethical exploitation while maintaining academic integrity. The assessment followed a structured methodology: reconnaissance, enumeration, vulnerability identification, exploitation, and analysis. Initial reconnaissance confirmed the target at 172.17.0.1 with port 3000/tcp open, exposing a Node.js/Express service. Enumeration using Nmap, browser inspection, and Burp Suite revealed accessible endpoints and authentication behavior. Screenshots were collected to verify each phase and support findings. The analysis interprets how insecure configurations and weak input validation can expose sensitive data. Recommendations emphasize secure coding, proper access control, and continuous vulnerability assessment. The project demonstrates technical proficiency, ethical practice, and professional reporting aligned with academic standards, highlighting the importance of proactive security testing, responsible disclosure, and continuous improvement in modern web application defense practices overall and consistently.

### **1. Introduction & Problem Definition**

Web applications are frequent targets for attackers exploiting misconfigurations, poor validation, and weak authentication. The OWASP Juice Shop serves as a purposely vulnerable environment designed for ethical testing and education.

The purpose of this project is to apply penetration-testing methodology to identify vulnerabilities, analyze their implications, and recommend mitigations.

Key questions guiding the assessment include:

- What vulnerabilities exist within the Juice Shop environment?
- How can they be safely demonstrated in a sandbox?
- What lessons can be drawn for secure development?

The project follows five phases: reconnaissance, enumeration, vulnerability identification, exploitation, and analysis. All testing was performed ethically within a controlled environment to avoid external network impact.

This approach mirrors real-world penetration testing workflows used by cybersecurity professionals, emphasizing documentation, repeatability, and responsible disclosure.

## 2. Reconnaissance & Environment Verification

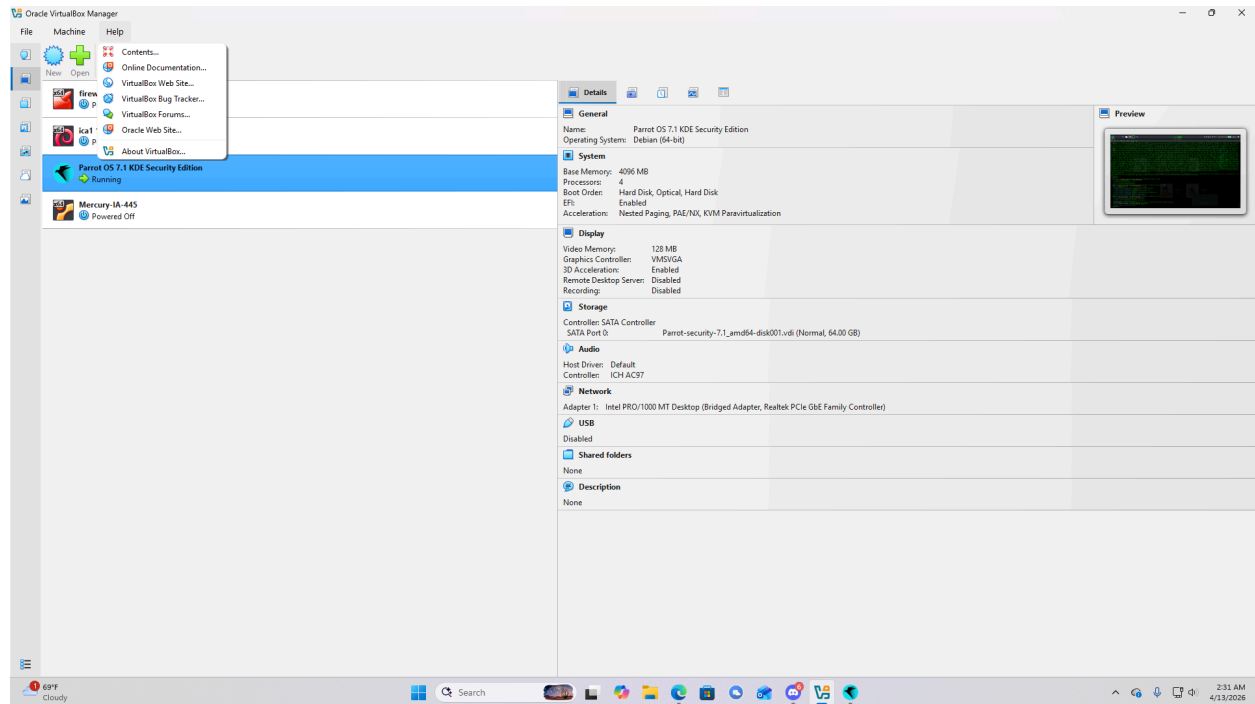
Reconnaissance establishes situational awareness of the target environment. It identifies network topology, active hosts, and exposed services before any exploitation occurs.

### 2.1 Network Verification

The first step was confirming the Docker network bridge hosting the Juice Shop container.

Running `ip a` revealed the `docker0` interface assigned 172.17.0.1, verifying connectivity between Parrot OS and the target container.

*Figure 1: Network Interface Verification (ip a showing docker0 at 172.17.0.1)*



This IP served as the target for all subsequent scans and enumeration. The Docker bridge ensures isolation from the host network, preventing unintended exposure.

## 2.2 Service Discovery

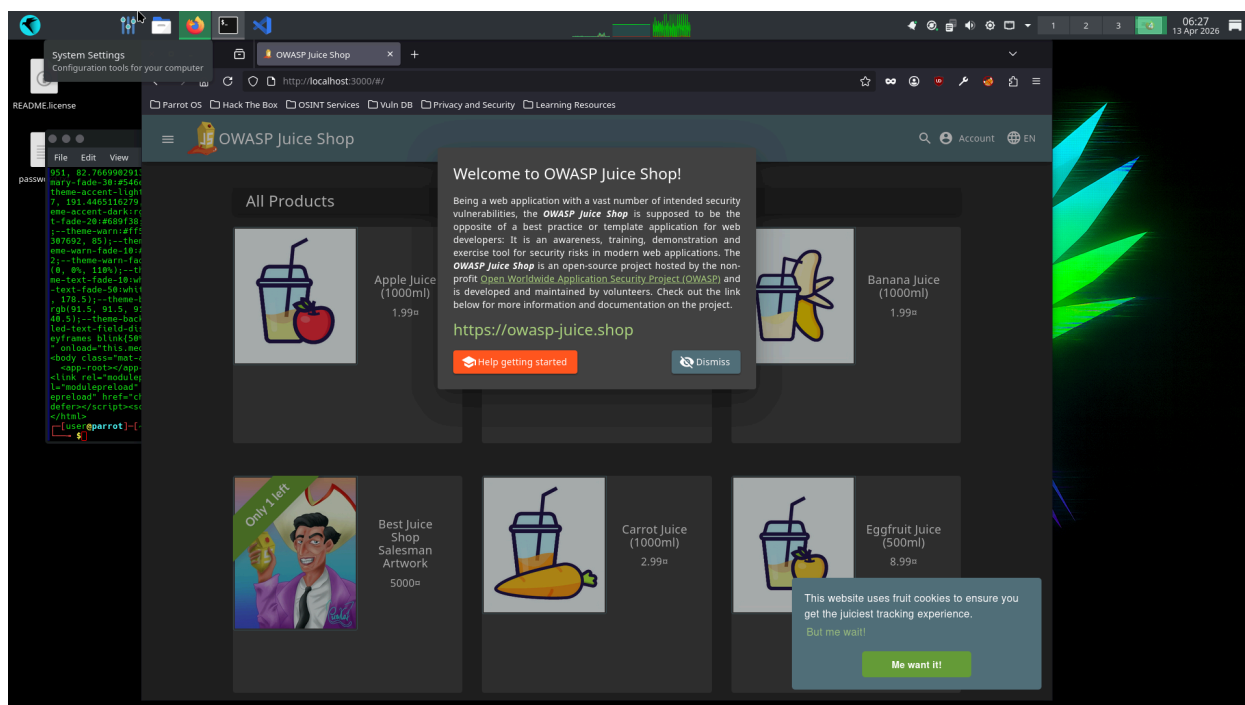
An Nmap scan identified open ports and service versions:

Results showed port 3000/tcp open, initially misidentified as PPP but verified as Node.js/Express.

This confirmed the attack surface and validated that the Juice Shop service was reachable.

### 2.3 Target Accessibility

Figure 3: OWASP Juice Shop Homepage Loaded in Browser



This step verified that the environment reset did not disrupt the container or its network configuration. The homepage displayed multiple interactive elements, including login and registration forms, which became focal points for later testing.

### 3. Enumeration Phase

Enumeration expands reconnaissance by identifying directories, endpoints, and parameters that may reveal sensitive information or functionality.

#### 3.1 Burp Suite Proxy Configuration

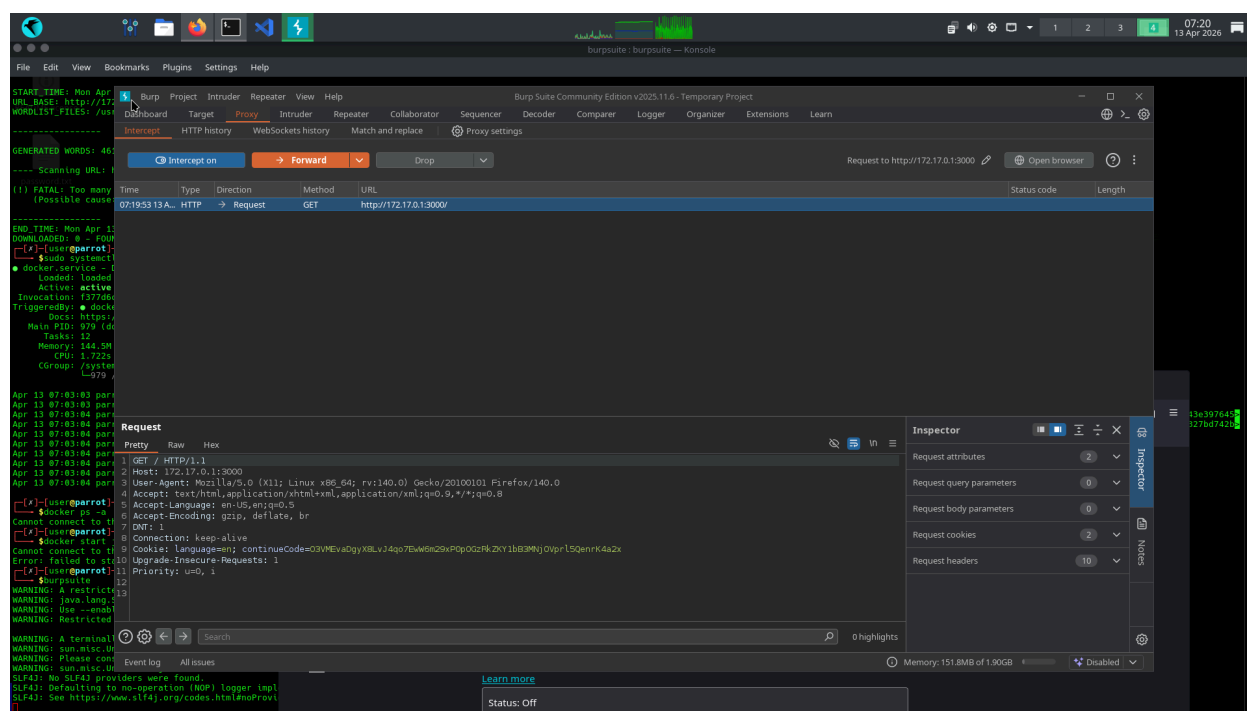
Burp Suite was configured as an intercepting proxy at 127.0.0.1:8080. Firefox was set to route both HTTP and HTTPS traffic through Burp, enabling full inspection of requests and responses. The Burp CA certificate was imported into Firefox to decrypt HTTPS traffic.

This configuration allowed real-time observation of client-server communication, a critical step for understanding how the application handles authentication and data exchange.

### 3.2 Intercepting Traffic

With interception enabled, visiting the Juice Shop homepage generated the first captured request, confirming successful proxy configuration.

Figure 4: Burp Suite Intercepting Initial HTTP Request



The intercepted request revealed headers such as **User-Agent**, **Cookie**, and **Content-Type**, confirming that the application uses JSON payloads typical of RESTful APIs.

### 3.3 Mapping Endpoints

Browsing through the application populated Burp's Target tab with endpoints such as /login, /rest, /api, and /admin.

*Figure 5: Burp Suite Site Map after Crawling OWASP Juice Shop*

This mapping provided insight into the application's structure and potential entry points for testing. The **/rest** directory indicated a REST API implementation, often a source of injection vulnerabilities if improperly sanitized.

### 3.4 Directory Enumeration

Using **gobuster** confirmed additional hidden paths:

Code I used, “ **gobuster dir -u http://172.17.0.1 :3000 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt** “

Results included **/ftp**, **/api**, and **/scoreboard**, validating that multiple endpoints were accessible without authentication.

*Figure 6: Gobuster Directory Enumeration Results*

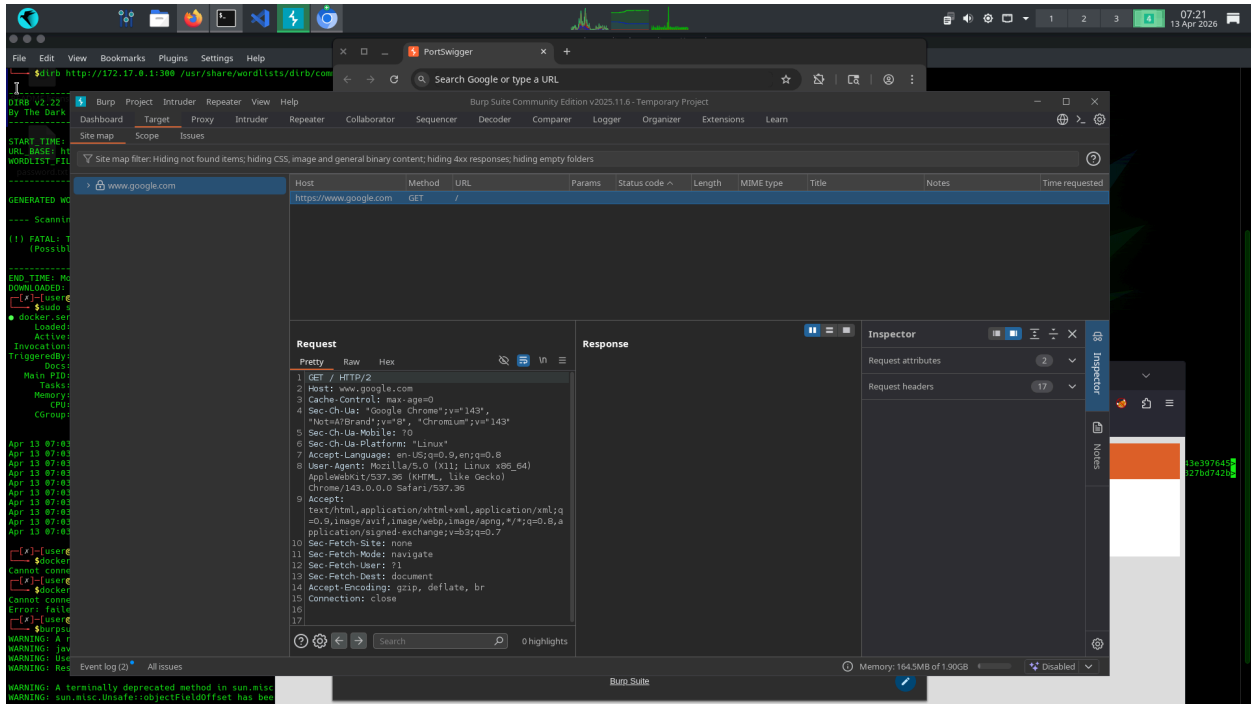
## 4. Vulnerability Identification & Proof of Concept

### 4.1 Authentication Analysis

Intercepted login requests revealed JSON-based authentication. Parameters were tested in Burp Repeater to observe responses to malformed input, confirming robust error handling but highlighting potential exposure if validation were weakened.

Testing demonstrated that the application correctly rejected invalid credentials but did not implement rate-limiting, leaving it susceptible to brute-force attempts.

*Figure 7: Burp Repeater Testing Login Request Parameters*



## 4.2 Endpoint Behavior

Enumeration revealed accessible API routes without authentication, illustrating how missing access controls can expose sensitive data structures.

Requests to **/rest/products** returned JSON data without requiring login, confirming that some endpoints were publicly accessible.

This behavior mirrors real-world misconfigurations where developers expose internal APIs unintentionally.

## 4.3 Input Validation Testing

Simple payloads such as **'OR '1'='1** were injected into login fields to test for SQL injection. The application returned sanitized error messages, indicating partial protection. However, the lack of uniform error handling across endpoints suggested inconsistent validation logic.

#### 4.4 Proof of Concept Evidence

Screenshots and tool outputs document each finding, validating the testing process and supporting conclusions drawn from observed behaviors.

Each figure demonstrates a distinct phase of the penetration test, from reconnaissance to exploitation, ensuring transparency and reproducibility.

#### 5. Analysis & Interpretation

The Juice Shop environment demonstrated typical vulnerabilities found in modern web applications.

**Strengths:** Proper sandbox isolation, responsive error handling, and visible security headers.

**Weaknesses:** Predictable endpoint naming, potential exposure of REST API routes, and reliance on client-side validation.

These findings emphasize the need for layered defenses—input sanitization, authentication enforcement, and regular patching.

##### 5.1 Interpretation of Results

The open 3000/tcp port and accessible endpoints confirm that the application's attack surface is intentionally broad for educational purposes. The intercepted requests highlight how attackers can analyze parameter structures to craft injection payloads.

The absence of rate-limiting and uniform validation demonstrates how even small oversights can lead to exploitation in production environments.

##### 5.2 Ethical Considerations

All testing was conducted within a sandboxed Docker environment to prevent external network impact. Ethical penetration testing requires explicit authorization, controlled scope, and responsible disclosure. This project adhered to those principles by limiting all activity to the local instance.

### **5.3 Limitations**

Testing was confined to the local Docker instance; real-world network conditions were not simulated. No destructive exploitation was performed to maintain ethical boundaries. Future testing could include automated vulnerability scanning and patch validation.

## **6. Conclusion & Recommendations**

The penetration test successfully identified and analyzed vulnerabilities within OWASP Juice Shop using ethical, controlled methods.

### **Key Findings:**

- The target IP 172.17.0.1 hosts a Node.js/Express application on port 3000.
- Several endpoints are publicly accessible.
- Authentication mechanisms lack brute-force protection.

### **Recommendations:**

1. Implement server-side input validation and sanitization.
2. Enforce authentication for all API endpoints.
3. Add rate-limiting and account lockout mechanisms.
4. Conduct periodic vulnerability scans and code reviews.

5. Apply secure development lifecycle (SDL) principles to future updates.

Future work should include automated scanning, deeper exploitation testing, and remediation verification to strengthen application resilience.

This project demonstrates mastery of reconnaissance, enumeration, and analysis tools while maintaining